Pandas 入门与数据处理实战手册

零基础入门版

竞赛培训组

2026 年 3 月 27 日

目录

1 Python 基础入门 **2**
1.1 什么是 Python? 2
1.2 安装 Python 2
1.3 验证 Python 安装 3
1.4 Python 基础语法 3
1.4.1 变量和数据类型 3
1.4.2 列表（List） 4
1.4.3 字典（Dictionary） 5
1.4.4 条件判断（if 语句） 5
1.4.5 循环（for 循环） 6
1.4.6 函数（Function） 7

2 准备工作：安装与导入 **9**
2.1 什么是 Pandas? 9
2.2 安装 Pandas 9
2.3 导入 Pandas 10

3 Pandas 的核心概念 **11**
3.1 DataFrame：数据表格 11
3.2 创建第一个 DataFrame 11

4 读取数据文件 **13**
4.1 为什么要从文件读取数据? 13
4.2 读取 CSV 文件 13
4.3 查看数据基本信息 14

5 数据选择：如何选取需要的数据 **15**
5.1 选择列 15
5.1.1 选择单列 16
5.1.2 选择多列 16
5.2 选择行 17
5.2.1 使用 iloc 选择行 17
5.2.2 使用 loc 选择行 17
5.3 同时选择行和列 18

6 数据清洗：处理脏数据 **19**
6.1 什么是脏数据？ 19
6.2 处理缺失值 19
6.2.1 识别缺失值 19
6.2.2 填充缺失值 20
6.2.3 删除缺失值 20
6.3 处理重复值 22
6.4 处理异常值 22
6.4.1 什么是异常值？ 22
6.4.2 检测异常值 22
6.4.3 处理异常值 23

7 数据转换与格式化 **24**
7.1 数据类型转换 24
7.2 字符串处理 24
7.3 应用函数 25

8 数据分组与统计 **27**
8.1 分组统计 27

9 综合实战：完整的数据处理流程 **28**
9.1 案例背景 28
9.2 分步实现 28

10 竞赛模拟题 **30**
10.1 题目一：电商订单数据处理 30
10.2 题目二：员工信息数据处理 30
10.3 题目一参考答案 31
10.4 题目二参考答案 36

11 常用代码速查表 **41**
11.1 数据读取与保存 41
11.2 数据查看 41
11.3 数据选择 41
11.4 数据清洗 42
11.5 数据转换 42
11.6 分组统计 43

1 Python 基础入门

1.1 什么是 Python?

知识点：Python 是什么

Python 是一种**编程语言**，就像中文、英语是人类交流的语言一样，Python 是人和计算机交流的语言。

Python 的特点：

- **简单易学**：语法接近自然语言，适合初学者
- **功能强大**：可以做数据分析、网站开发、人工智能等
- **应用广泛**：大数据、机器学习、自动化办公等领域都在用

为什么要学 Python?

- 数据处理必备技能
- 竞赛中必须使用的工具
- 就业市场需求大

1.2 安装 Python

在使用 Pandas 之前，需要先安装 Python。

步骤：

1. 访问官网：https://www.python.org/downloads/
2. 下载最新版本的 Python（建议 3.8 或以上）
3. 运行安装程序，**一定要勾选"Add Python to PATH"**
4. 点击"Install Now" 完成安装

安装注意事项

- 安装时务必勾选"Add Python to PATH"，否则后面会有问题
- 如果已经安装了 Python，可以跳过这一步
- 安装完成后，重启电脑确保环境变量生效

1.3 验证 Python 安装

安装完成后，验证一下是否安装成功：

1. 按 Win + R 键，输入 cmd，回车打开命令行
2. 输入以下命令：

```
python --version
```

3. 如果显示版本号（如 Python 3.9.0），说明安装成功

1.4 Python 基础语法

1.4.1 变量和数据类型

知识点：什么是变量？

变量就像是给数据起个名字，方便我们以后使用。

类比理解：

- 变量名 = 标签
- 变量值 = 盒子里的东西

比如：age = 20，就是把数字 20 放进一个叫 age 的盒子里。

```python
# 定义变量（给数据起名字）
name = "张三"        # 字符串（文字）
age = 20            # 整数（数字）
height = 1.75       # 浮点数（小数）
is_student = True # 布尔值（真/假）

# 打印变量
print(name)
print(age)
print(height)
print(is_student)
```

Listing 1: 变量示例

Python 常用数据类型：

表 1: Python 数据类型

类型	说明	示例
int	整数	20, 100, -5
float	浮点数（小数）	1.75, 3.14
str	字符串（文字）	” 张三”, ”hello”
bool	布尔值（真/假）	True, False
list	列表（一组数据）	[1, 2, 3]
dict	字典（键值对）	{”name”: ” 张三”}

1.4.2 列表（List）

知识点：什么是列表？

列表就是一组数据的集合，用方括号 `[]` 表示。

类比理解：

- 列表 = 购物清单、成绩单、通讯录
- 可以存放多个数据
- 数据之间有顺序

```python
# 创建一个列表
scores = [85, 92, 78, 90, 88]
names = ["张三", "李四", "王五"]

# 访问列表元素（索引从0开始）
print(scores[0])     # 第一个元素: 85
print(scores[1])     # 第二个元素: 92
print(names[2])      # 第三个元素: 王五

# 获取列表长度
print(len(scores))   # 5

# 添加元素
scores.append(95)    # 在末尾添加95
```

Listing 2: 列表示例

1.4.3 字典（Dictionary）

知识点：什么是字典？

字典是一种键值对的数据结构，用花括号 {} 表示。

类比理解：

- 字典 = 真实字典（查词找释义）
- 键（key）= 要查的词
- 值（value）= 词的释义

```python
# 创建一个字典
student = {
    "name": "张三",
    "age": 20,
    "score": 85
}

# 访问字典元素
print(student["name"])    # 张三
print(student["age"])     # 20

# 修改值
student["score"] = 90

# 添加新键值对
student["gender"] = "男"
```

Listing 3: 字典示例

1.4.4 条件判断（if 语句）

```python
# 条件判断
age = 18

if age >= 18:
    print("成年人")
else:
    print("未成年人")
```

```python

# 多条件判断
score = 85

if score >= 90:
    print("优秀")
elif score >= 80:
    print("良好")
elif score >= 60:
    print("及格")
else:
    print("不及格")
```

Listing 4: 条件判断示例

知识点：条件判断的语法

- `if`：如果条件成立
- `elif`：否则如果（可以有多个）
- `else`：否则（前面都不成立时执行）
- **注意**：Python 用**缩进**表示代码块，不是大括号

1.4.5 循环（for 循环）

```python
# 遍历列表
scores = [85, 92, 78, 90]

for score in scores:
    print(score)

# 遍历范围
for i in range(5):      # 0, 1, 2, 3, 4
    print(i)

# 遍历字典
student = {"name": "张三", "age": 20}

for key, value in student.items():
```

```
    print(f"{key}: {value}")
```

Listing 5: 循环示例

1.4.6 函数（Function）

知识点：什么是函数？

函数就是一段可以重复使用的代码块。

类比理解：

- 函数 = 洗衣机、电饭煲（封装好的功能）
- 输入 = 脏衣服、米（参数）
- 输出 = 干净衣服、米饭（返回值）

好处：

- 避免重复写代码
- 代码更清晰
- 便于维护

```
# 定义函数
def greet(name):
    """打招呼函数"""
    return f"你好，{name}！"

# 调用函数
result = greet("张三")
print(result)    # 你好，张三！

# 带多个参数的函数
def calculate_total(price, quantity):
    """计算总价"""
    return price * quantity

# 调用函数
total = calculate_total(100, 3)
print(total)     # 300
```

Listing 6: 函数示例

动手练习：Python 基础

完成以下练习，巩固 Python 基础：

练习 1：变量和类型

```python
# 创建变量
name = "你的名字"
age = 你的年龄
height = 你的身高

# 打印这些信息
print(f"姓名：{name}")
print(f"年龄：{age}")
print(f"身高：{height}米")
```

练习 2：列表操作

```python
# 创建一个成绩列表
scores = [78, 85, 92, 67, 88]

# 计算平均分
total = sum(scores)
count = len(scores)
average = total / count

print(f"总分：{total}")
print(f"平均分：{average}")
```

练习 3：条件判断

```python
# 判断成绩等级
score = 85

if score >= 90:
    level = "优秀"
elif score >= 80:
    level = "良好"
```

```python
elif score >= 60:
    level = "及格"
else:
    level = "不及格"

print(f"成绩等级: {level}")
```

2 准备工作：安装与导入

2.1 什么是 Pandas?

知识点：Pandas 是什么

Pandas 是 Python 的一个数据分析库，它的名字来源于"Panel Data"（面板数据）。简单来说，Pandas 就是用来处理表格数据的工具，就像 Excel 一样，但比 Excel 更强大、更灵活。

为什么叫 Pandas?

- 和熊猫（panda）没有关系
- 来自计量经济学的"Panel Data" 概念
- 是 Python Data Analysis 的缩写

2.2 安装 Pandas

在使用 Pandas 之前，需要先安装它。打开命令行（CMD 或 PowerShell），输入以下命令：

```python
# 使用pip安装（推荐）
pip install pandas

# 如果安装了Anaconda，使用conda安装
conda install pandas
```

Listing 7: 安装 Pandas

安装小贴士

- 安装时需要联网
- 如果安装速度慢，可以使用国内镜像源
- 安装完成后，关闭命令行重新打开

2.3 导入 Pandas

安装完成后，在 Python 代码中导入 Pandas：

```python
# 导入pandas库，并给它起别名pd
import pandas as pd

# 同时导入numpy（经常和pandas一起使用）
import numpy as np
```

Listing 8: 导入 Pandas

知识点：为什么要用别名？

在 Python 中，给库起别名是为了：

1. **简化代码**：写 pd 比写 pandas 更短
2. **行业惯例**：大家都用 pd，方便阅读
3. **保持一致**：不同人写的代码风格统一

这就好比：

- 你的名字叫”张三丰”，但朋友们叫你”小张”
- 写代码时，pandas 是全名，pd 是小名

3 Pandas 的核心概念

3.1 DataFrame：数据表格

知识点：什么是 DataFrame?

DataFrame 是 Pandas 中最重要的数据结构，你可以把它想象成一张 Excel 表格：

- 有行和列
- 每列可以是不同的数据类型（数字、文字、日期等）
- 有列名（表头）
- 每行有一个索引（行号）

类比理解：

- Excel 表格 → DataFrame
- 工作表 → DataFrame
- 数据库表 → DataFrame

3.2 创建第一个 DataFrame

让我们创建一个简单的学生成绩表：

```python
import pandas as pd

# 创建一个字典（dict），存储数据
data = {
    '姓名': ['张三', '李四', '王五'],
    '数学': [85, 92, 78],
    '英语': [88, 85, 92]
}

# 用字典创建DataFrame
df = pd.DataFrame(data)

# 显示DataFrame
print(df)
```

Listing 9: 创建 DataFrame

运行结果：

	姓名	数学	英语
0	张三	85	88
1	李四	92	85
2	王五	78	92

知识点：代码拆解

让我们逐行理解上面的代码：

1. `data = {...}`：创建一个字典
   - 字典的键（'姓名'、'数学'）变成列名
   - 字典的值（列表）变成列的数据
2. `pd.DataFrame(data)`：把字典转换成 DataFrame
3. `print(df)`：显示表格内容
   - 最左边一列 0、1、2 是**行索引**（默认从 0 开始）
   - 上面一行是**列名**

动手练习 1

试着创建一个包含以下信息的 DataFrame：

- 3 个商品：手机、电脑、耳机
- 每个商品有：名称、价格、库存数量

参考代码：

```python
import pandas as pd

data = {
    '商品名称': ['手机', '电脑', '耳机'],
    '价格': [2999, 5999, 199],
    '库存': [100, 50, 200]
}

df = pd.DataFrame(data)
print(df)
```

4 读取数据文件

4.1 为什么要从文件读取数据？

在实际工作中，数据通常存储在文件中，而不是直接写在代码里。常见的数据文件格式有：

- **CSV 文件**（.csv）：逗号分隔的值，最常用
- **Excel 文件**（.xlsx）：Excel 表格
- **JSON 文件**（.json）：网页数据常用格式

4.2 读取 CSV 文件

CSV（Comma-Separated Values）是最常见的数据格式。假设我们有一个文件学生成绩.csv：

```python
import pandas as pd

# 读取CSV文件
df = pd.read_csv('学生成绩.csv')

# 显示前5行
print(df.head())
```

Listing 10: 读取 CSV 文件

知识点：read_csv 常用参数

pd.read_csv() 有很多参数，常用的有：

- encoding='utf-8'：指定文件编码（处理中文）
- sep=','：指定分隔符（默认是逗号）
- header=0：指定第几行是列名（默认第 0 行）

示例：

```python
# 读取中文CSV文件（防止乱码）
df = pd.read_csv('学生成绩.csv', encoding='utf-8')

# 读取用制表符分隔的文件
```

```python
df = pd.read_csv('数据.txt', sep='\t')
```

中文乱码怎么办？

如果读取后中文显示乱码，尝试以下编码：

- `encoding='utf-8'`：最常用的编码
- `encoding='gbk'`：中文 Windows 系统常用
- `encoding='gb2312'`：简体中文编码

4.3 查看数据基本信息

读取数据后，首先要了解数据的基本情况：

```python
# 查看数据形状（行数，列数）
print(df.shape)

# 查看列名
print(df.columns)

# 查看数据类型
print(df.dtypes)

# 查看基本信息（包括非空值数量）
print(df.info())

# 查看统计摘要（只针对数值列）
print(df.describe())
```

Listing 11: 查看数据信息

知识点：这些方法的作用

- `shape`：返回（行数，列数），比如（100，5）表示 100 行 5 列
- `columns`：返回所有列名
- `dtypes`：返回每列的数据类型

- info()：显示详细信息，包括每列的非空值数量
- describe()：统计数值列的计数、均值、最值等

动手练习 2

使用提供的 01_ 电商订单数据 _ 练习.csv 文件：

1. 读取这个 CSV 文件
2. 查看数据有多少行、多少列
3. 查看所有列名
4. 查看每列的数据类型

参考代码：

```python
import pandas as pd

# 读取数据
df = pd.read_csv('01_电商订单数据_练习.csv')

# 查看形状
print("数据形状:", df.shape)

# 查看列名
print("\n列名:")
print(df.columns)

# 查看数据类型
print("\n数据类型:")
print(df.dtypes)
```

5 数据选择：如何选取需要的数据

5.1 选择列

有时候我们只需要表格中的某些列，而不是全部。

5.1.1 选择单列

```
# 选择'姓名'这一列
names = df['姓名']

# 或者使用点号（仅适用于列名没有空格和特殊字符时）
names = df.姓名
```

Listing 12: 选择单列

知识点：单列选择的结果

选择单列后，得到的是一个 **Series**（序列），不是 DataFrame。

Series 和 DataFrame 的区别：

- DataFrame：二维表格（有行有列）
- Series：一维数组（只有一列数据）

5.1.2 选择多列

```
# 选择多列（用两个方括号）
subset = df[['姓名', '数学']]

# 注意：外层[]是DataFrame的索引，内层[]是Python列表
```

Listing 13: 选择多列

注意双括号

选择多列时，需要用**两个方括号**：

- 外层 []：DataFrame 的列选择
- 内层 []：Python 的列表，包含要选的列名

常见错误：

```
# 错误！只用一个方括号
df['姓名', '数学']  # 会报错

# 正确！用两个方括号
```

```
df[['姓名', '数学']]  # OK
```

5.2 选择行

Pandas 提供了两种选择行的方法：

知识点：loc vs iloc

- loc：按**标签**选择（用行名）
- iloc：按**位置**选择（用行号，从 0 开始）

类比：

- loc 就像用姓名找人
- iloc 就像用学号找人（第 1 个、第 2 个...）

5.2.1 使用 iloc 选择行

```
# 选择第1行（索引为0）
row = df.iloc[0]

# 选择前3行
rows = df.iloc[0:3]

# 选择第1、3、5行
rows = df.iloc[[0, 2, 4]]
```

Listing 14: 使用 iloc 选择行

5.2.2 使用 loc 选择行

```
# 假设行索引是姓名
df_indexed = df.set_index('姓名')

# 选择姓名为'张三'的行
row = df_indexed.loc['张三']

```

```
# 选择多个人的数据
rows = df_indexed.loc[['张三', '李四']]
```

Listing 15: 使用 loc 选择行

5.3 同时选择行和列

```
# iloc[行位置, 列位置]
# 选择前3行, 前2列
data = df.iloc[0:3, 0:2]

# loc[行标签, 列标签]
# 选择'张三'和'李四'的'数学'和'英语'成绩
data = df_indexed.loc[['张三', '李四'], ['数学', '英语']]
```

Listing 16: 同时选择行和列

动手练习 3

使用 01_ 电商订单数据 _ 练习.csv:

1. 只选择 product_name 和 price 两列
2. 选择前 10 行数据
3. 选择第 5 到第 15 行的 order_id、user_id、quantity 三列

参考代码:

```
import pandas as pd

df = pd.read_csv('01_电商订单数据_练习.csv')

# 1. 选择两列
product_info = df[['product_name', 'price']]
print(product_info.head())

# 2. 选择前10行
first_10 = df.iloc[0:10]
print(first_10)

```

```python
# 3. 选择特定行和列
subset = df.iloc[4:15][['order_id', 'user_id', 'quantity']]
print(subset)
```

6 数据清洗：处理脏数据

6.1 什么是脏数据？

知识点：常见的数据质量问题

真实世界的数据往往存在各种问题，我们称之为" 脏数据"：

1. **缺失值**：某些单元格是空的
2. **重复值**：同样的数据出现多次
3. **异常值**：明显不合理的数据（如年龄为-5）
4. **格式不一致**：日期格式混乱、大小写不统一
5. **类型错误**：数字被存成了文字

6.2 处理缺失值

6.2.1 识别缺失值

```python
# 检查每列的缺失值数量
print(df.isnull().sum())

# 检查缺失值比例
missing_ratio = df.isnull().sum() / len(df) * 100
print(missing_ratio)
```

Listing 17: 检查缺失值

知识点：isnull() 的作用

`isnull()` 方法会返回一个和原表格一样大的表格，里面只有 True 和 False：

- True：表示这个单元格是缺失值

- False：表示这个单元格有数据

`sum()` 会把 True 当成 1、False 当成 0，所以能统计出每列的缺失值数量。

6.2.2 填充缺失值

有几种常见的填充策略：

```
# 方法1：用固定值填充
df['年龄'] = df['年龄'].fillna(0)

# 方法2：用平均值填充（适合数值列）
df['成绩'] = df['成绩'].fillna(df['成绩'].mean())

# 方法3：用中位数填充（适合有异常值的情况）
df['工资'] = df['工资'].fillna(df['工资'].median())

# 方法4：用众数填充（适合类别数据）
df['性别'] = df['性别'].fillna(df['性别'].mode()[0])

# 方法5：用前一个值填充（适合时间序列）
df['温度'] = df['温度'].fillna(method='ffill')
```

Listing 18: 填充缺失值的方法

选择填充策略的建议

- **数值数据**：优先用中位数（对异常值不敏感）
- **类别数据**：用众数（出现次数最多的值）
- **时间序列**：用前后值填充（保持趋势）
- **缺失太多**（如超过 50%）：考虑删除该列

6.2.3 删除缺失值

如果缺失值很少，可以直接删除：

```
# 删除包含缺失值的行
df_clean = df.dropna()

```

```python
# 只删除某列有缺失值的行
df_clean = df.dropna(subset=['年龄'])

# 删除整列都是缺失值的列
df_clean = df.dropna(axis=1, how='all')
```

Listing 19: 删除缺失值

动手练习 4

使用 03_ 缺失值处理 _ 练习.csv:

1. 统计每列的缺失值数量
2. 用中位数填充 salary 列的缺失值
3. 用众数填充 department 列的缺失值
4. 删除 score 列仍有缺失值的行

参考代码:

```python
import pandas as pd

df = pd.read_csv('03_缺失值处理_练习.csv')

# 1. 统计缺失值
print("缺失值统计:")
print(df.isnull().sum())

# 2. 用中位数填充工资
median_salary = df['salary'].median()
df['salary'] = df['salary'].fillna(median_salary)
print(f"\n用 {median_salary} 填充工资缺失值")

# 3. 用众数填充部门
mode_dept = df['department'].mode()[0]
df['department'] = df['department'].fillna(mode_dept)
print(f"用 '{mode_dept}' 填充部门缺失值")

# 4. 删除score仍有缺失的行
df = df.dropna(subset=['score'])
```

```python
print(f"\n删除后剩余 {len(df)} 行")
```

6.3 处理重复值

```python
# 检查重复值
duplicates = df.duplicated()
print(f"有 {duplicates.sum()} 行重复")

# 查看重复的行
duplicate_rows = df[df.duplicated(keep=False)]
print(duplicate_rows)

# 删除重复值（保留第一个）
df_clean = df.drop_duplicates()

# 基于特定列删除重复
df_clean = df.drop_duplicates(subset=['姓名'])
```

Listing 20: 处理重复值

6.4 处理异常值

6.4.1 什么是异常值？

异常值是指明显偏离正常范围的数据，比如：

- 年龄为负数或超过 150
- 价格为 0 或异常高
- 数量为负数

6.4.2 检测异常值

```python
# IQR（四分位距）方法
def detect_outliers_iqr(data):
    Q1 = data.quantile(0.25)  # 第25百分位数
    Q3 = data.quantile(0.75)  # 第75百分位数
    IQR = Q3 - Q1
```

```python

    # 异常值范围
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # 找出异常值
    outliers = data[(data < lower_bound) | (data > upper_bound)]
    return outliers

# 检测价格的异常值
price_outliers = detect_outliers_iqr(df['price'])
print(f"发现 {len(price_outliers)} 个价格异常值")
```

Listing 21: 使用 IQR 方法检测异常值

知识点：IQR 方法原理

IQR（Interquartile Range，四分位距）是一种稳健的异常值检测方法：

1. 把数据从小到大排序
2. 找到第 25% 位置的数（Q1）和第 75% 位置的数（Q3）
3. 计算 IQR = Q3 - Q1
4. 正常数据范围：[Q1 - 1.5×IQR, Q3 + 1.5×IQR]
5. 超出这个范围的就是异常值

优点：不受极端值影响，适合偏态分布的数据

6.4.3 处理异常值

找到异常值后，有几种处理方式：

```python
# 方法1：删除异常值
df_clean = df[(df['price'] >= lower_bound) & (df['price'] <= upper_bound)]

# 方法2：用边界值替换（截断）
df['price'] = df['price'].clip(lower=lower_bound, upper=upper_bound)

```

```python
# 方法3：用中位数替换
median_price = df['price'].median()
df.loc[df['price'] > upper_bound, 'price'] = median_price
```

Listing 22: 处理异常值的方法

7 数据转换与格式化

7.1 数据类型转换

```python
# 查看当前数据类型
print(df.dtypes)

# 转换为整数
df['年龄'] = df['年龄'].astype(int)

# 转换为浮点数
df['价格'] = df['价格'].astype(float)

# 转换为字符串
df['编号'] = df['编号'].astype(str)

# 转换为日期时间
df['日期'] = pd.to_datetime(df['日期'])
```

Listing 23: 转换数据类型

7.2 字符串处理

```python
# 去除空格
df['姓名'] = df['姓名'].str.strip()

# 转换大小写
df['邮箱'] = df['邮箱'].str.lower()

# 替换字符
df['电话'] = df['电话'].str.replace('-', '')

```

```python
# 提取子串
df['年份'] = df['日期'].str[0:4]

# 分割字符串
df['姓名'].str.split(' ', expand=True)
```

Listing 24: 常用字符串方法

知识点：str 访问器

在 Pandas 中，要对字符串列进行操作，需要使用.str 访问器：

- df[' 列名'].str. 方法 ()
- 不能直接写 df[' 列名'].strip()

这就像打开一个工具箱，.str 就是打开字符串工具箱的钥匙。

7.3 应用函数

有时候需要对数据进行复杂的转换，可以自定义函数：

```python
# 定义一个函数
def score_level(score):
    if score >= 90:
        return '优秀'
    elif score >= 80:
        return '良好'
    elif score >= 60:
        return '及格'
    else:
        return '不及格'

# 应用到整列
df['等级'] = df['成绩'].apply(score_level)

# 使用lambda函数（简单操作）
df['价格_打折'] = df['价格'].apply(lambda x: x * 0.8)
```

Listing 25: 使用 apply 应用函数

动手练习 5

使用 04_ 数据格式清洗 _ 练习.csv：

1. 去除 name 列的前后空格
2. 将 income 列中的"k" 替换为"000"，并转换为数值
3. 从 birth_date 列中提取年份

参考代码：

```python
import pandas as pd

df = pd.read_csv('04_数据格式清洗_练习.csv')

# 1. 去除空格
df['name'] = df['name'].str.strip()

# 2. 处理收入（将12k转换为12000）
def convert_income(income_str):
    income_str = str(income_str)
    if 'k' in income_str.lower():
        number = float(income_str.lower().replace('k', ''))
        return number * 1000
    else:
        return float(income_str)

df['income_num'] = df['income'].apply(convert_income)

# 3. 提取年份
df['birth_year'] = df['birth_date'].str[0:4]

print(df[['name', 'income', 'income_num', 'birth_year']].
    head())
```

8 数据分组与统计

8.1 分组统计

知识点：分组统计的概念

分组统计就是：

1. 先把数据按某个列分成若干组
2. 然后对每组进行统计计算

类比：

- 把全班同学按**性别**分组，计算每组平均成绩
- 把商品按**类别**分组，计算每类总销量

```python
# 按部门分组，计算平均工资
dept_avg = df.groupby('部门')['工资'].mean()
print(dept_avg)

# 多列分组，多统计量
df.groupby(['部门', '性别']).agg({
    '工资': ['mean', 'max', 'min'],
    '年龄': 'mean'
})
```

Listing 26: 分组统计

知识点：groupby 的工作流程

`groupby()` 的工作流程就像流水线：

1. **分组**：按指定列的值把数据分成多组
2. **应用**：对每组应用统计函数
3. **合并**：把各组结果合并成一个表格

常用统计函数：

- `mean()`：平均值
- `sum()`：求和

- count()：计数
- max()/min()：最大/最小值
- std()：标准差

9 综合实战：完整的数据处理流程

9.1 案例背景

假设我们有一个电商订单数据集，需要完成以下任务：

1. 清洗数据（处理缺失值、重复值、异常值）
2. 转换数据格式（价格、日期、手机号）
3. 分析数据（统计各品类销售额）

9.2 分步实现

```python
import pandas as pd
import numpy as np

# ========== 第1步：读取数据 ==========
print("步骤1：读取数据")
df = pd.read_csv('01_电商订单数据_练习.csv')
print(f"原始数据：{df.shape[0]}行，{df.shape[1]}列")

# ========== 第2步：查看数据质量 ==========
print("\n步骤2：数据质量检查")
print("缺失值统计：")
print(df.isnull().sum())
print(f"\n重复值：{df.duplicated().sum()}行")

# ========== 第3步：处理重复值 ==========
print("\n步骤3：删除重复值")
df = df.drop_duplicates()
print(f"删除后：{df.shape[0]}行")

# ========== 第4步：处理缺失值 ==========
```

```python
print("\n步骤4: 处理缺失值")
# user_id缺失用'UNKNOWN'填充
df['user_id'] = df['user_id'].fillna('UNKNOWN')

# ========== 第5步: 处理异常值 ==========
print("\n步骤5: 处理异常值")
# 处理负数数量
df.loc[df['quantity'] < 0, 'quantity'] = 1
print("负数数量已修正为1")

# ========== 第6步: 格式转换 ==========
print("\n步骤6: 格式转换")
# 提取价格数字
df['price_num'] = df['price'].str.replace('元', '').astype(float
    )

# 标准化手机号
df['phone_clean'] = df['phone'].str.replace('-', '')

# ========== 第7步: 数据分析 ==========
print("\n步骤7: 数据分析")
# 计算订单金额
df['total_amount'] = df['price_num'] * df['quantity']

# 按商品统计
product_stats = df.groupby('product_name').agg({
    'total_amount': 'sum',
    'quantity': 'sum'
}).sort_values('total_amount', ascending=False)

print("\n销售额TOP5商品: ")
print(product_stats.head())

# ========== 第8步: 保存结果 ==========
print("\n步骤8: 保存结果")
df.to_csv('清洗后数据.csv', index=False)
print("数据已保存到: 清洗后数据.csv")
```

Listing 27: 完整数据处理流程

10 竞赛模拟题

10.1 题目一：电商订单数据处理

竞赛模拟题 1

题目要求：

使用 `01_电商订单数据_练习.csv`，完成以下任务：

1. 读取数据，查看基本信息
2. 删除重复订单
3. 处理 user_id 缺失值（用'UNKNOWN' 填充）
4. 处理负数数量（修正为 1）
5. 提取价格中的数字，转换为数值型
6. 标准化手机号格式（去除横线）
7. 计算每个订单的总金额（价格 × 数量）
8. 统计每个商品的总销售额
9. 保存清洗后的数据

输出要求：

- 清洗后的数据文件
- 数据统计报告（包含处理记录和统计结果）

10.2 题目二：员工信息数据处理

竞赛模拟题 2

题目要求：

使用 `03_缺失值处理_练习.csv` 和 `04_数据格式清洗_练习.csv`，完成以下任务：

1. 读取两个 CSV 文件
2. 处理缺失值：
   - age：用中位数填充

- salary：用平均值填充
- department：用众数填充

3. 格式标准化：
   - 姓名：去除前后空格
   - 收入：统一转换为数值（处理"k" 单位）
4. 按部门统计平均工资和年龄
5. 保存处理后的数据

10.3 题目一参考答案

竞赛模拟题 1 - 完整解答

解题思路：

这道题要求我们对电商订单数据进行全面清洗和分析。按照数据处理的正常流程，我们应该：

1. 先读取数据，了解数据的基本情况
2. 处理数据质量问题（重复、缺失、异常）
3. 进行格式转换
4. 进行数据分析
5. 保存结果

完整代码：

```python
import pandas as pd
import numpy as np

# ========== 第1步：读取数据 ==========
print("=" * 50)
print("步骤1：读取数据")
print("=" * 50)

df = pd.read_csv('01_电商订单数据_练习.csv')
print(f"数据形状：{df.shape}")
print(f"\n列名：{df.columns.tolist()}")
```

```python
print("\n前5行数据: ")
print(df.head())

# ========== 第2步: 删除重复值 ==========
print("\n" + "=" * 50)
print("步骤2: 删除重复值")
print("=" * 50)

# 查看重复值
n_duplicates = df.duplicated().sum()
print(f"发现 {n_duplicates} 行重复数据")

# 删除重复值
df = df.drop_duplicates()
print(f"删除后剩余 {len(df)} 行")

# ========== 第3步: 处理缺失值 ==========
print("\n" + "=" * 50)
print("步骤3: 处理缺失值")
print("=" * 50)

# 查看缺失值
print("缺失值统计: ")
print(df.isnull().sum())

# 填充user_id缺失值
df['user_id'] = df['user_id'].fillna('UNKNOWN')
print("\nuser_id缺失值已用'UNKNOWN'填充")

# ========== 第4步: 处理异常值 ==========
print("\n" + "=" * 50)
print("步骤4: 处理异常值")
print("=" * 50)

# 查看数量列的情况
print("数量统计: ")
print(df['quantity'].describe())

```

```python
# 处理负数数量
negative_count = (df['quantity'] < 0).sum()
print(f"\n发现 {negative_count} 个负数数量")

# 将负数改为1
df.loc[df['quantity'] < 0, 'quantity'] = 1
print("负数数量已修正为1")

# ========== 第5步：提取价格数字 ==========
print("\n" + "=" * 50)
print("步骤5：提取价格数字")
print("=" * 50)

# 查看价格列的格式
print("价格示例：")
print(df['price'].head())

# 提取数字（去除'元'字）
df['price_num'] = df['price'].str.replace('元', '').astype(
    float)

print("\n转换后的价格：")
print(df[['price', 'price_num']].head())

# ========== 第6步：标准化手机号 ==========
print("\n" + "=" * 50)
print("步骤6：标准化手机号")
print("=" * 50)

# 查看手机号格式
print("手机号示例：")
print(df['phone'].head())

# 去除横线
df['phone_clean'] = df['phone'].str.replace('-', '')

print("\n清洗后的手机号：")
print(df[['phone', 'phone_clean']].head())
```

```python

# ========== 第7步：计算订单金额 ==========
print("\n" + "=" * 50)
print("步骤7：计算订单金额")
print("=" * 50)

# 计算总金额 = 价格 × 数量
df['total_amount'] = df['price_num'] * df['quantity']

print("订单金额计算完成")
print(df[['product_name', 'price_num', 'quantity', 'total_amount']].head())

# ========== 第8步：统计商品销售额 ==========
print("\n" + "=" * 50)
print("步骤8：统计商品销售额")
print("=" * 50)

# 按商品分组统计
product_sales = df.groupby('product_name').agg({
    'total_amount': 'sum',       # 总销售额
    'quantity': 'sum',           # 总销量
    'order_id': 'count'          # 订单数
}).sort_values('total_amount', ascending=False)

product_sales.columns = ['总销售额', '总销量', '订单数']

print("商品销售额TOP10：")
print(product_sales.head(10))

# ========== 第9步：生成报告并保存 ==========
print("\n" + "=" * 50)
print("步骤9：生成报告并保存")
print("=" * 50)

# 生成处理报告
report = f"""
数据清洗报告
```

```
{'=' * 50}
1. 原始数据: 50行, 9列
2. 删除重复值: {n_duplicates}行
3. 处理后数据: {len(df)}行
4. 处理内容:
   - user_id缺失值用'UNKNOWN'填充
   - 负数数量修正为1
   - 价格转换为数值型
   - 手机号去除横线
5. 统计结果:
   - 商品种类: {df['product_name'].nunique()}种
   - 总销售额: {df['total_amount'].sum():.2f}元
   - 总订单数: {len(df)}单
{'=' * 50}
"""

print(report)

# 保存报告到文件
with open('数据处理报告.txt', 'w', encoding='utf-8') as f:
    f.write(report)

print("报告已保存到: 数据处理报告.txt")

# 保存清洗后的数据
df.to_csv('清洗后订单数据.csv', index=False, encoding='utf-8')
print("数据已保存到: 清洗后订单数据.csv")

print("\n" + "=" * 50)
print("数据处理完成！")
print("=" * 50)
```

Listing 28: 电商订单数据处理完整解答

关键步骤说明：

1. **读取数据：** 使用 `pd.read_csv()` 读取 CSV 文件
2. **查看信息：** 用 `shape`、`columns`、`head()` 了解数据

3. **删除重复**：duplicated() 检测，drop_duplicates() 删除
4. **填充缺失**：fillna() 用指定值填充
5. **处理异常**：用条件筛选找到异常值，然后修正
6. **字符串处理**：str.replace() 去除不需要的字符
7. **类型转换**：astype(float) 转换为数值型
8. **计算新列**：直接用列相乘生成新列
9. **分组统计**：groupby() 配合 agg() 进行多维度统计

10.4 题目二参考答案

竞赛模拟题 2 - 完整解答

解题思路：

这道题涉及两个文件的处理，需要：

1. 分别读取两个 CSV 文件
2. 对缺失值采用不同的填充策略
3. 对格式混乱的数据进行标准化
4. 合并分析结果

完整代码：

```python
import pandas as pd
import numpy as np

# ========== 第1步：读取数据 ==========
print("=" * 50)
print("步骤1：读取数据")
print("=" * 50)

# 读取两个CSV文件
df1 = pd.read_csv('03_缺失值处理_练习.csv')
df2 = pd.read_csv('04_数据格式清洗_练习.csv')

print(f"员工数据：{df1.shape}")
```

```python
print(f"格式清洗数据: {df2.shape}")

# ========== 第2步: 处理缺失值 (df1) ==========
print("\n" + "=" * 50)
print("步骤2: 处理缺失值")
print("=" * 50)

# 查看缺失值情况
print("缺失值统计: ")
print(df1.isnull().sum())

# 2.1 age用中位数填充
age_median = df1['age'].median()
df1['age'] = df1['age'].fillna(age_median)
print(f"\nage缺失值用中位数 {age_median} 填充")

# 2.2 salary用平均值填充
salary_mean = df1['salary'].mean()
df1['salary'] = df1['salary'].fillna(salary_mean)
print(f"salary缺失值用平均值 {salary_mean:.2f} 填充")

# 2.3 department用众数填充
dept_mode = df1['department'].mode()[0]
df1['department'] = df1['department'].fillna(dept_mode)
print(f"department缺失值用众数 '{dept_mode}' 填充")

# 验证缺失值是否处理完毕
print("\n处理后缺失值统计: ")
print(df1.isnull().sum())

# ========== 第3步: 格式标准化 (df2) ==========
print("\n" + "=" * 50)
print("步骤3: 格式标准化")
print("=" * 50)

# 3.1 去除姓名前后空格
print("姓名处理前示例: ")
print(df2['name'].head())
```

```python

df2['name'] = df2['name'].str.strip()

print("\n姓名处理后示例: ")
print(df2['name'].head())

# 3.2 处理收入格式（将12k转换为12000）
print("\n收入处理前示例: ")
print(df2['income'].head(10))

def convert_income(income_str):
    """将收入转换为数值"""
    # 转换为字符串
    income_str = str(income_str).strip()

    # 如果包含k或K，表示千元
    if 'k' in income_str.lower():
        # 提取数字部分
        number = float(income_str.lower().replace('k', ''))
        return number * 1000
    else:
        # 直接转换为浮点数
        return float(income_str)

# 应用转换函数
df2['income_num'] = df2['income'].apply(convert_income)

print("\n收入处理后示例: ")
print(df2[['income', 'income_num']].head(10))

# ========== 第4步: 按部门统计 ==========
print("\n" + "=" * 50)
print("步骤4: 按部门统计")
print("=" * 50)

# 4.1 员工数据统计
dept_stats = df1.groupby('department').agg({
    'salary': ['mean', 'max', 'min'],
```

```python
    'age': 'mean'
}).round(2)

# 重命名列
dept_stats.columns = ['平均工资', '最高工资', '最低工资', '平均年龄']

print("各部门统计: ")
print(dept_stats)

# 4.2 计算部门人数
dept_count = df1['department'].value_counts()
print("\n部门人数: ")
print(dept_count)

# ========== 第5步: 生成完整报告 ==========
print("\n" + "=" * 50)
print("步骤5: 生成报告")
print("=" * 50)

report = f"""
员工数据处理报告
{'=' * 50}

一、数据概况
  员工数据: {len(df1)}人
  部门数量: {df1['department'].nunique()}个

二、缺失值处理
  age: 用中位数 {age_median} 填充
  salary: 用平均值 {salary_mean:.2f} 填充
  department: 用众数 '{dept_mode}' 填充

三、格式标准化
  姓名: 去除前后空格
  收入: 统一转换为数值（处理k单位）

四、部门统计
```

```
{dept_stats.to_string()}

五、部门人数
{dept_count.to_string()}
{'=' * 50}
"""

print(report)

# 保存报告
with open('员工数据处理报告.txt', 'w', encoding='utf-8') as f:
    f.write(report)

print("报告已保存到: 员工数据处理报告.txt")

# ========== 第6步: 保存处理后的数据 ==========
print("\n" + "=" * 50)
print("步骤6: 保存数据")
print("=" * 50)

# 保存员工数据
df1.to_csv('处理后员工数据.csv', index=False, encoding='utf-8')
print("员工数据已保存到: 处理后员工数据.csv")

# 保存格式清洗数据
df2.to_csv('处理后格式数据.csv', index=False, encoding='utf-8')
print("格式数据已保存到: 处理后格式数据.csv")

print("\n" + "=" * 50)
print("数据处理完成！")
print("=" * 50)
```

Listing 29: 员工信息数据处理完整解答

关键步骤说明：

1. **读取多个文件：**分别用 `pd.read_csv()` 读取

2. **不同填充策略：**
   - 数值型（age）：用 `median()` 中位数，不受异常值影响
   - 数值型（salary）：用 `mean()` 平均值
   - 分类型（department）：用 `mode()` 众数
3. **自定义函数：**用 `apply()` 应用自定义转换函数
4. **多维度统计：**用 `agg()` 同时进行多个统计计算
5. **生成报告：**将处理过程和结果整理成文本报告

11 常用代码速查表

11.1 数据读取与保存

```
# 读取CSV
df = pd.read_csv('文件.csv', encoding='utf-8')

# 保存CSV
df.to_csv('输出.csv', index=False, encoding='utf-8')
```

Listing 30: 读写数据

11.2 数据查看

```
df.head()           # 前5行
df.tail()           # 后5行
df.shape            # 形状（行数，列数）
df.columns          # 列名
df.dtypes           # 数据类型
df.info()           # 详细信息
df.describe()       # 统计摘要
```

Listing 31: 查看数据

11.3 数据选择

```python
# 选择列
df['列名']            # 单列
df[['列1', '列2']]   # 多列

# 选择行
df.iloc[0]            # 第1行（按位置）
df.iloc[0:5]          # 前5行
df.loc['标签']        # 按标签

# 条件筛选
df[df['年龄'] > 18]
df[(df['年龄'] > 18) & (df['性别'] == '男')]
```

Listing 32: 选择数据

11.4 数据清洗

```python
# 缺失值
df.isnull().sum()                          # 统计缺失值
df.dropna()                                # 删除缺失值
df['列'].fillna(值)                        # 填充缺失值
df['列'].fillna(df['列'].mean())          # 用均值填充

# 重复值
df.duplicated().sum()                      # 统计重复值
df.drop_duplicates()                       # 删除重复值

# 异常值（IQR方法）
Q1 = df['列'].quantile(0.25)
Q3 = df['列'].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df['列'] < Q1-1.5*IQR) | (df['列'] > Q3+1.5*IQR)]
```

Listing 33: 清洗数据

11.5 数据转换

```python
# 类型转换
```

```python
df['列'].astype(int)
df['列'].astype(str)
pd.to_datetime(df['日期列'])

# 字符串处理
df['列'].str.strip()                          # 去空格
df['列'].str.replace('旧', '新')              # 替换
df['列'].str.contains('关键词')               # 包含判断

# 应用函数
df['新列'] = df['列'].apply(函数)
```

Listing 34: 转换数据

11.6 分组统计

```python
# 简单分组
df.groupby('分组列')['统计列'].mean()

# 多列多统计
df.groupby(['列1', '列2']).agg({
    '列3': ['mean', 'sum'],
    '列4': 'count'
})
```

Listing 35: 分组统计